- **相交链表**
  - 链接：相交链表
  - 描述

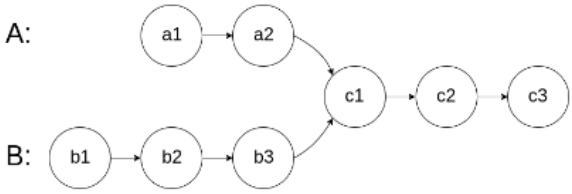
160. 相交链表

已解答

简单 相关标签 相关企业 A文

给你两个单链表的头节点 `headA` 和 `headB` ，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 `null` 。

图示两个链表在节点 `c1` 开始相交：



题目数据 **保证** 整个链式结构中不存在环。

注意，函数返回结果后，链表必须 **保持其原始结构** 。

  - 答案

```
class ListNode {
  val: number
  next: ListNode | null
  constructor(val?: number, next?: ListNode | null) {
    this.val = val === undefined ? 0 : val
    this.next = next === undefined ? null : next
  }
}

function getIntersectionNode(headA: ListNode | null, headB: ListNode | null): ListNode | null {
  let pA = headA
  let pB = headB
  while (pA !== pB) {
    pA = pA === null ? headB : pA.next
    pB = pB === null ? headA : pB.next
  }
  return pA
}
```

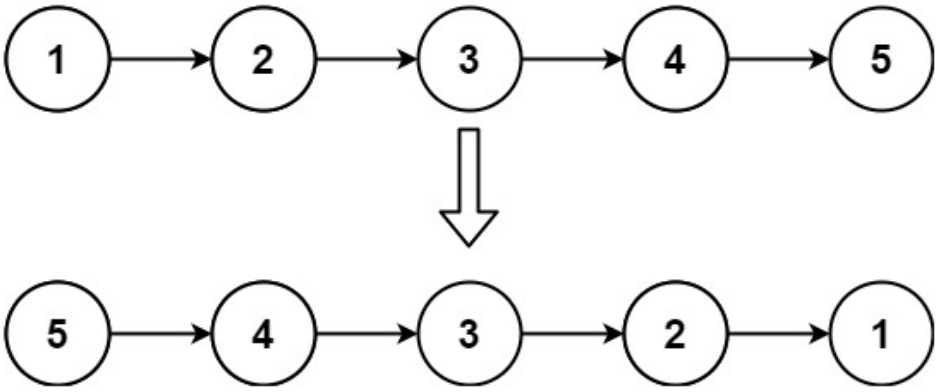
- **反转链表**
  - 链接：反转链表
  - 描述

206. 反转链表

简单 相关标签 相关企业 A文

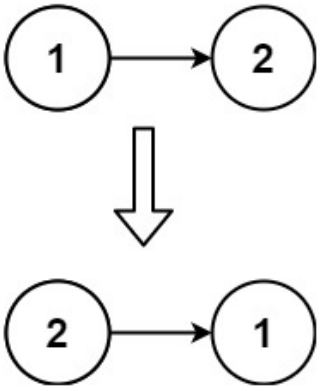
给你单链表的头节点 `head` ，请你反转链表，并返回反转后的链表。

示例 1：



> **输入：** head = [1,2,3,4,5]
> **输出：** [5,4,3,2,1]

示例 2：



> **输入：** head = [1,2]
> **输出：** [2,1]

示例 3：

> **输入：** head = []
> **输出：** []

提示：

- 链表中节点的数目范围是 `[0, 5000]`
- `-5000 <= Node.val <= 5000`

- 答案

```
class ListNode {
```

```
  val: number
  next: ListNode | null
  constructor(val?: number, next?: ListNode | null) {
    this.val = val === undefined ? 0 : val
    this.next = next === undefined ? null : next
  }
}

function reverseList(head: ListNode | null): ListNode | null {
  let prev = null
  let current = head
  while (current) {
    const next = current.next
    current.next = prev
    prev = current
    current = next
  }
  return prev
}
```

- **环形链表**
  - 链接：环形链表
  - 描述

141. 环形链表

已解答

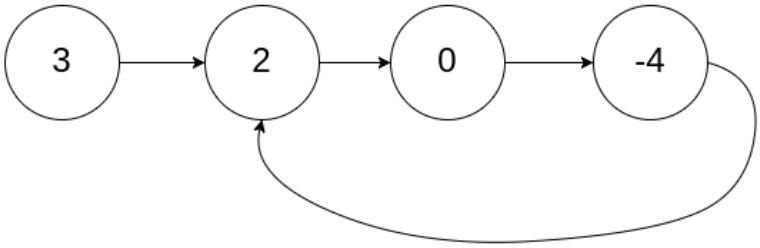
简单 相关标签 相关企业

给你一个链表的头节点 `head` ，判断链表中是否有环。

如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。 为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（索引从 0 开始）。**注意：** `pos` **不作为参数进行传递** 。仅仅是为了标识链表的实际情况。

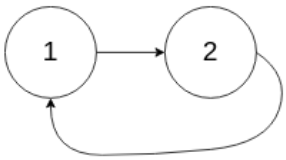
如果链表中存在环 ，则返回 `true` 。 否则，返回 `false` 。

示例 1：



> **输入：** head = [3,2,0,-4], pos = 1
> **输出：** true
> **解释：** 链表中有一个环，其尾部连接到第二个节点。

示例 2：



> **输入：** head = [1,2], pos = 0
> **输出：** true
> **解释：** 链表中有一个环，其尾部连接到第一个节点。

示例 3：

> **输入：** head = [1], pos = -1
> **输出：** false
> **解释：** 链表中没有环。

提示：

- 链表中节点的数目范围是 $[0, 10^4]$
- $-10^5 <= Node.val <= 10^5$
- `pos` 为 `-1` 或者链表中的一个 **有效索引** 。

- 答案

```
class ListNode {
  val: number
  next: ListNode | null
  constructor(val?: number, next?: ListNode | null) {
    this.val = val === undefined ? 0 : val
```

```
    this.next = next === undefined ? null : next
  }
}

function hasCycle(head: ListNode | null): boolean {
  let slow = head
  let fast = head?.next ?? null
  while (slow !== fast) {
    if (fast === null) {
      return false
    }
    slow = slow?.next ?? null
    fast = fast.next?.next ?? null
  }
  return slow !== null // 不直接返回true，目的是处理首次head为null的特殊情况
}
```

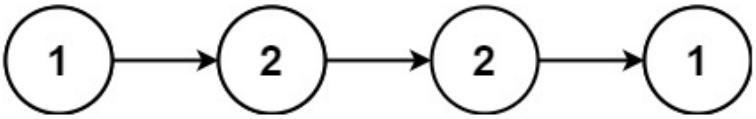
- **回文链表**
  - 链接：回文链表
  - 描述

234. 回文链表

简单 相关标签 相关企业 A文

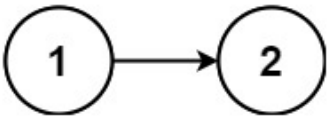
给你一个单链表的头节点 `head` ，请你判断该链表是否为回文链表。如果是，返回 `true` ；否则，返回 `false` 。

示例 1：



> **输入：** head = [1,2,2,1]
> **输出：** true

示例 2：



> **输入：** head = [1,2]
> **输出：** false

提示：

- 链表中节点数目在范围 $[1, 10^5]$ 内
- `0 <= Node.val <= 9`

  - 答案

```
class ListNode {
  val: number
  next: ListNode | null
  constructor(val?: number, next?: ListNode | null) {
    this.val = val === undefined ? 0 : val
    this.next = next === undefined ? null : next
  }
}

// 不能用栈的思路，因为没法处理链表节点为奇数的情况
function isPalindrome(head: ListNode | null): boolean {
  const valueList: number[] = []
  let node = head
  while (node) {
    valueList.push(node.val)
```

```
    node = node.next
  }
  for (let i = 0, j = valueList.length - 1; i < j; i++, j--) {
    if (valueList[i] !== valueList[j]) {
      return false
    }
  }
  return true
}
```

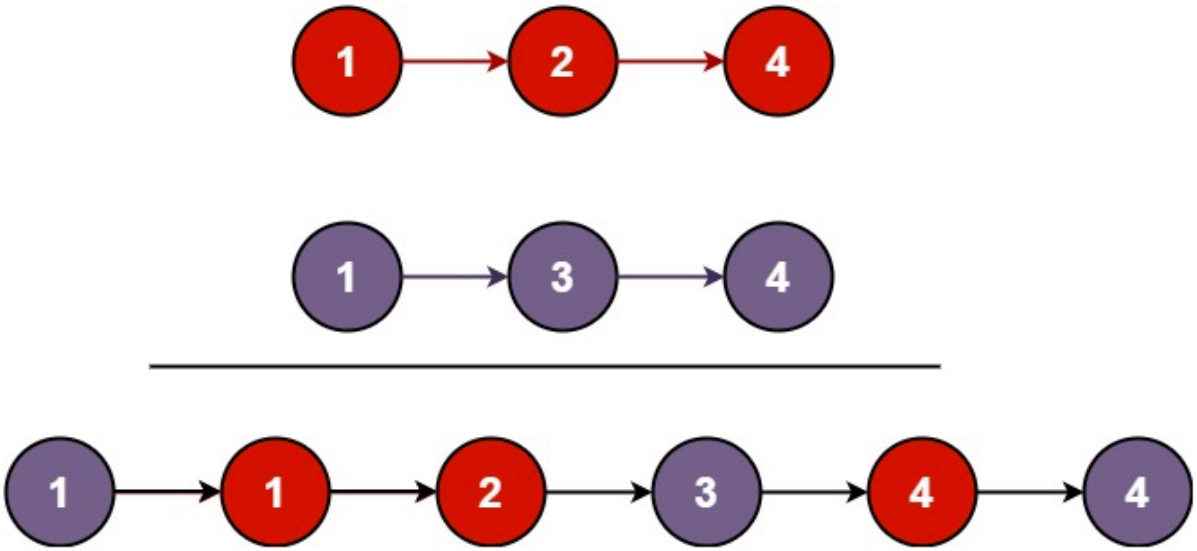
- **合并两个有序链表**
  - 链接：合并两个有序链表
  - 描述

21. 合并两个有序链表

简单 相关标签 相关企业 A文

将两个升序链表合并为一个新的 **升序** 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1：



> **输入：** l1 = [1,2,4], l2 = [1,3,4]
> **输出：** [1,1,2,3,4,4]

示例 2：

> **输入：** l1 = [], l2 = []
> **输出：** []

示例 3：

> **输入：** l1 = [], l2 = [0]
> **输出：** [0]

提示：

- 两个链表的节点数目范围是 `[0, 50]`
- `-100 <= Node.val <= 100`
- `l1` 和 `l2` 均按 **非递减顺序** 排列
  - 答案

```
class ListNode {
  val: number
  next: ListNode | null
  constructor(val?: number, next?: ListNode | null) {
    this.val = val === undefined ? 0 : val
```

```
    this.next = next === undefined ? null : next
  }
}

function mergeTwoLists(list1: ListNode | null, list2: ListNode | null): ListNode | null {
  if (list1 === null) {
    return list2
  } else if (list2 === null) {
    return list1
  } else if (list1.val < list2.val) {
    list1.next = mergeTwoLists(list1.next, list2)
    return list1
  } else {
    list2.next = mergeTwoLists(list2.next, list1)
    return list2
  }
}
```